Gibbs' Reflective Cycle Self-Reflection Report

Course: CP3406 / CP5307 Mobile Computing

Assessment: Assessment 1 — Utility App

App Name: AuraWeather At-a-Glance Utility App

Student Name: Mobile Developer Student

Word Count: ~850 words

1. Description (What Happened?)

For Assessment 1, I designed and developed **AuraWeather**, an "at-a-glance" Android utility application aimed at providing immediate weather conditions and contextual outdoor recommendations for university campus life (focusing on James Cook University campuses in Townsville and Cairns).

The application was built using modern Android standards introduced in Weeks 1–5 of the course:

- **Language & Framework**: Kotlin with Jetpack Compose declarative UI.
- **Design System**: Material Design 3 (M3) incorporating elevated cards, segmented control buttons, filter chips, and dynamic surface colors.
- **Architecture**: Model-View-ViewModel (MVVM) pattern utilizing `ViewModel` and reactive `StateFlow` streams.
- **Data & Networking**: The Repository Pattern abstraction coupled with Retrofit 2 and Gson converter to consume real-time REST API data from Open-Meteo.
- **Settings & Navigation**: Single-activity navigation using Material 3 `Scaffold` and `NavigationBar` to switch seamlessly between `UtilityScreen` and `SettingsScreen`.

In addition to building the core UI and networking layers, I authored unit tests for the ViewModel and Repository, created Compose UI previews, and maintained structured documentation.

2. Feelings (What Were Your Feelings and Reactions?)

At the beginning of the assignment, I felt enthusiastic about creating a utility app with real-world usefulness, yet slightly anxious regarding the integration of Retrofit and asynchronous coroutines within Jetpack Compose state management. Having previously built basic static layouts, moving to asynchronous web network calls raised concerns about handling screen rotations, loading delays, and potential API errors gracefully.

During the initial phase of wiring Retrofit with `StateFlow`, encountering asynchronous state transitions initially felt overwhelming. However, as I successfully mapped the API response into domain objects and saw live weather data from JCU Townsville render inside my Compose cards, my confidence grew significantly. I felt a strong sense of accomplishment when the settings screen toggles dynamically updated the units and card visibility on the main utility screen in real-time.

3. Evaluation (What Went Well and What Didn't?)

What Went Well:

- **Clean Architecture Implementation**: Separating data classes (`WeatherData`, `City`, `UserPreferences`), network DTOs (`WeatherResponse`), `WeatherRepository`, and `WeatherViewModel` ensured clean code separation and made unit testing effortless.
- **At-a-Glance User Experience**: The hero weather card and the outdoor recommendation callout successfully delivered rapid, intuitive information without cluttering the screen.
- **API Choice**: Selecting the Open-Meteo API proved to be ideal because it required no complex API keys, returned reliable JSON responses, and provided rich metrics including humidity, wind speed, and WMO weather codes.
- **Jetpack Compose Modularity**: Reusable composables such as `MetricCard`, `ErrorCard`, and `ToggleRow` allowed for clean, maintainable UI code.

What Didn't Go So Well:

- **State Synchronization Initial Hiccup**: Initially, updating user preferences on the settings screen did not immediately reflect on the main utility screen because the preferences state was isolated. I resolved this by unifying preferences inside the ViewModel's state container and re-emitting updated UI states.
- **Toolchain & Version Alignment**: Transitioning Java toolchain versions from Java 11 to Java 17 initially caused Gradle preview build warnings, requiring explicit configuration in `build.gradle.kts`.

4. Analysis (Why Did Things Go Well or Not Well?)

Things went well primarily due to adhering strictly to recommended Android architecture patterns (MVVM and Repository pattern). Using `StateFlow` provided a predictable, single source of truth for UI state. When a user changed a location or unit preference in `SettingsScreen`, the `WeatherViewModel` updated `_preferences` and triggered re-fetching or re-formatting of data cleanly.

The challenges with state synchronization occurred because I initially attempted to manage state locally inside composables rather than hoisting state up to the ViewModel. Once I applied Jetpack Compose state hoisting principles and collected state via `collectAsState()`, the UI became completely reactive.

5. Conclusion (What Did You Learn?)

Through this project, I gained deep practical insight into modern Android application development:

1. **Asynchronous UI State**: I learned how to represent UI states using Kotlin sealed interfaces (`Loading`, `Success`, `Error`) and how Compose automatically re-composes when `StateFlow` emits new values.
2. **Repository Pattern Benefits**: Decoupling Retrofit API service calls behind a `WeatherRepository` interface allowed me to mock network responses in unit tests without making actual HTTP requests during testing.
3. **Utility App UX Principles**: I learned that a good utility app prioritizes high-priority information at the top of the hierarchy, reducing cognitive load and friction for the user.

6. Action Plan (What Would You Do Differently Next Time?)

If I were to undertake a similar project in the future, I would:

1. **Incorporate DataStore Persistence**: While Assessment 1 did not require persistent settings, storing user preferences in `androidx.datastore` would ensure settings persist across app restarts.
2. **Adopt Dependency Injection Framework**: I would implement **Hilt** or **Koin** for dependency injection rather than manual instantiation, making dependency management even cleaner.
3. **Offline Caching**: Implement a Room database or local cache so that weather data is instantly displayed offline before refreshing from the network.